

Unix/Linux - The vi Editor

vi Editor of Unix

- No matter what work you do with the UNIX system, you'll eventually write some C programs or shell (or perl) scripts. You may have to edit some of the system files at times. For all of this you must learn to use an editor, and UNIX provides a very old and versatile one—vi. Bill Joy created this editor for the BSD system.
- Editing files using the screen-oriented text editor vi is one of the best ways.
- This editor enables you to edit lines in context with other lines in the file.
- An improved version of the vi editor which is called the VIM is also available.
- Here, VIM stands for Vi IMproved.

vi Editor of Unix

- vi is generally considered the de facto standard in Unix editors because
 - It's usually available on all the flavors of Unix system.
 - Its implementations are very similar across the board.
 - It requires very few resources.
 - It is more user-friendly than other editors such as the **ed** or the **ex**.
- You can use the **vi** editor to edit an existing file or to create a new file from scratch.
- You can also use this editor to just read a text file.

Starting the vi Editor

- The basic commands to use the vi editor
 - **vi filename**
 - Creates a new file if it already does not exist, otherwise opens an existing file.
 - **vi -R filename**
 - Opens an existing file in the read-only mode.
 - **view filename**
 - Opens an existing file in the read-only mode.

vi Editor: Example

- To create a new file **testfile** if it already does not exist in the current working directory write

```
$vi testfile
```

- The above command will generate the following output

```
|
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
~
```

```
"testfile" [New File]
```

vi Editor: Example

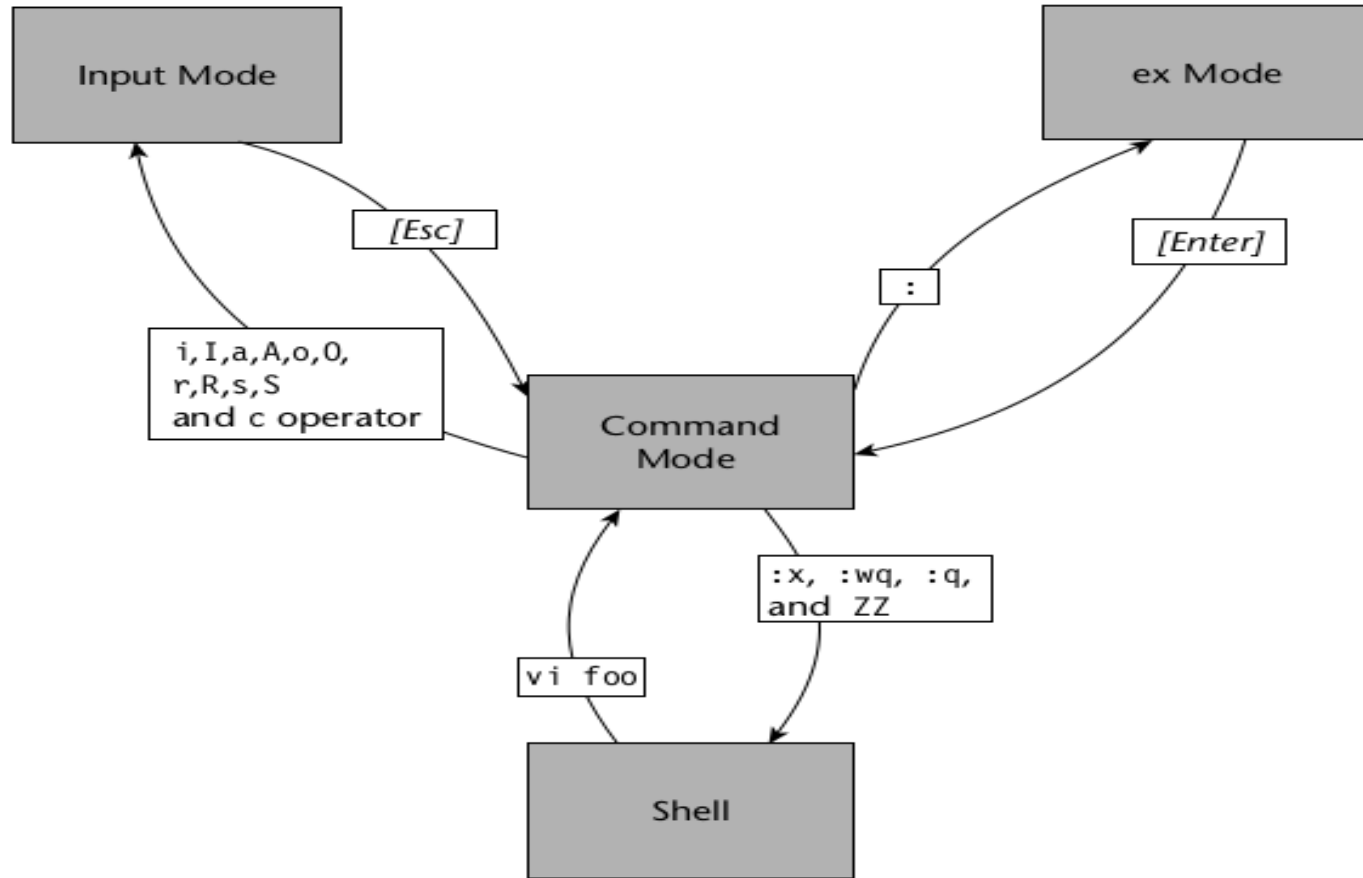
- You will notice a **tilde** (~) on each line following the cursor.
A tilde represents an unused line.
- If a line does not begin with a tilde and appears to be blank, there is a space, tab, newline, or some other non-viewable character present.
- You now have one open file to start working on.

Operation Modes

- The vi editor usually operates in the following three modes:
 - **Command Mode**—The default mode where every key pressed is interpreted as a command to run on text. Navigation, copying, and deleting text are performed in this mode. You used h and l in this mode to move the cursor along a line.
 - **Input Mode**— This mode is invoked by pressing one of the keys. Every key pressed subsequently shows up as text. Pressing [Esc] in this mode takes vi to Command Mode. We used i as an Input Mode command.
 - **ex Mode or Last Line Mode**—This mode is used for file handling and performing substitution. vi was originally hard-linked to a line editor named ex; that's where this mode got its name from. A : in the Command Mode invokes this mode, and is then followed by an ex Mode command. vi normally switches to the Command Mode after the ex Mode command is run, though we used :x to save the buffer and quit vi.
- vi always starts in the **command mode**. To enter text, you must be in the insert mode for which simply type **i**. To come out of the insert mode, press the **Esc** key, which will take you back to the command mode.

Hint – If you are not sure which mode you are in, press the Esc key twice; this will take you to the command mode.

The Three Modes



Getting Out of vi

- The command to quit out of vi is **:q**. Once in the command mode, type colon, and 'q', followed by return.
- The command to quit out of vi without saving is **:q!**. This lets you exit vi without saving any of the changes.
- The command to save the contents of the editor is **:w**
- You can combine the above command with the quit command, or use **:wq** and return.
- The easiest way to **save your changes and exit vi** is with the **ZZ** command. When you are in the command mode, type **ZZ**. The **ZZ** command works the same way as the **:wq** command.
- If you want to specify/state any particular name for the file, you can do so by specifying it after the **:w**. For example, if you wanted to save the file you were working on as another filename called **filename2**, you would type **:w filename2** and return.

Moving within a File

- To move around within a file without affecting your text, you must be in the command mode (press Esc twice). The following table lists out a few commands you can use to move around one character at a time:
 - **k** Moves the cursor up one line
 - **j** Moves the cursor down one line
 - **h** Moves the cursor to the left one character position
 - **l** Moves the cursor to the right one character position
- The following points need to be considered to move within a file
 - **vi is case-sensitive. You need to pay attention to capitalization when using the commands.**
 - **Most commands in vi can be prefaced by the number of times you want the action to occur. For example, **2j** moves the cursor two lines down the cursor location.**

Move Command in VI Editor

There are many other ways to move within a file in vi. Remember that you must be in the command mode (**press Esc twice**).

0 or	Positions the cursor at the beginning of a line
\$	Positions the cursor at the end of a line
W	Positions the cursor to the next word
b	Positions the cursor to the previous word
(	Positions the cursor to the beginning of the current sentence
)	Positions the cursor to the beginning of the next sentence
E	Moves to the end of the blank delimited word
{	Moves a paragraph back
}	Moves a paragraph forward
[[	Moves a section back
]]	Moves a section forward

Move Command in VI Editor

0 or	Positions the cursor at the beginning of a line
\$	Positions the cursor at the end of a line
W	Positions the cursor to the next word
b	Positions the cursor to the previous word
(	Positions the cursor to the beginning of the current sentence
)	Positions the cursor to the beginning of the next sentence
E	Moves to the end of the blank delimited word
{	Moves a paragraph back
}	Moves a paragraph forward
[[	Moves a section back
]]	Moves a section forward

Few tips

- Make use of the repeat factor. vi also supports the use of a repeat factor as a command prefix to repeat the command as many times as the prefix. So if the Command Mode command `k` moves the cursor up one line, then `10k` moves it up 10 lines. Use the repeat factor wherever you can to speed up operations.
- Undo whenever you make a mistake. If you have made a mistake in editing, either by wrongly deleting text or inserting it at an incorrect location, then as a first measure, just press `[Esc]` and then `u` to undo the last action. If that makes matters worse, use `u` again. Linux users should instead use `[Ctrl-r]`.
- Use `[Ctrl-L]` to clear the screen. If the screen gets garbled, enter this control sequence in the Command Mode to redraw the screen. If you hit `[Ctrl-L]` in the Input Mode, you'll see the symbol `^L` on the screen. Use the backspace key to wipe it out, press `[Esc]`, and then hit `[Ctrl-L]`.
- Don't use `[CapsLock]` on most occasions. vi commands are case-sensitive; `a` and `A` are different Input Mode commands. Even if you have to activate `[CapsLock]` to enter a block of text in uppercase, make sure you deactivate it after the text entry is complete.
- Don't use the PC navigation keys Avoid using the keys marked Up, Down, Left, and Right, `[PageUp]` and `[PageDown]`. Many of them could fail when you use vi over a network connection. vi provides an elaborate set of keys for navigation purposes.

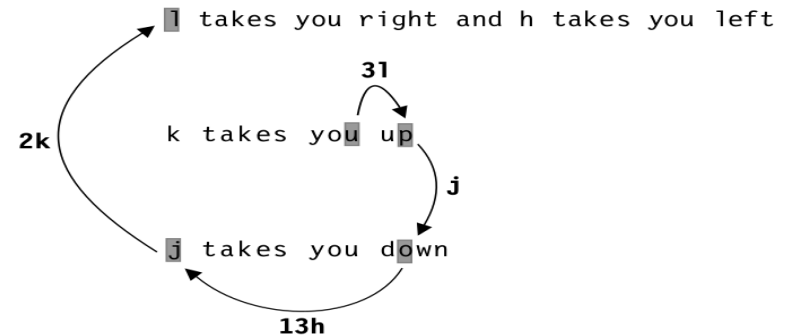
Word navigation

Command	Function
Relative Motion	
h (or [Backspace])	Moves cursor left
l (or spacebar)	Moves cursor right
5l	Moves five characters right
k	Moves cursor up
10k	Moves 10 lines up
j	Moves cursor down

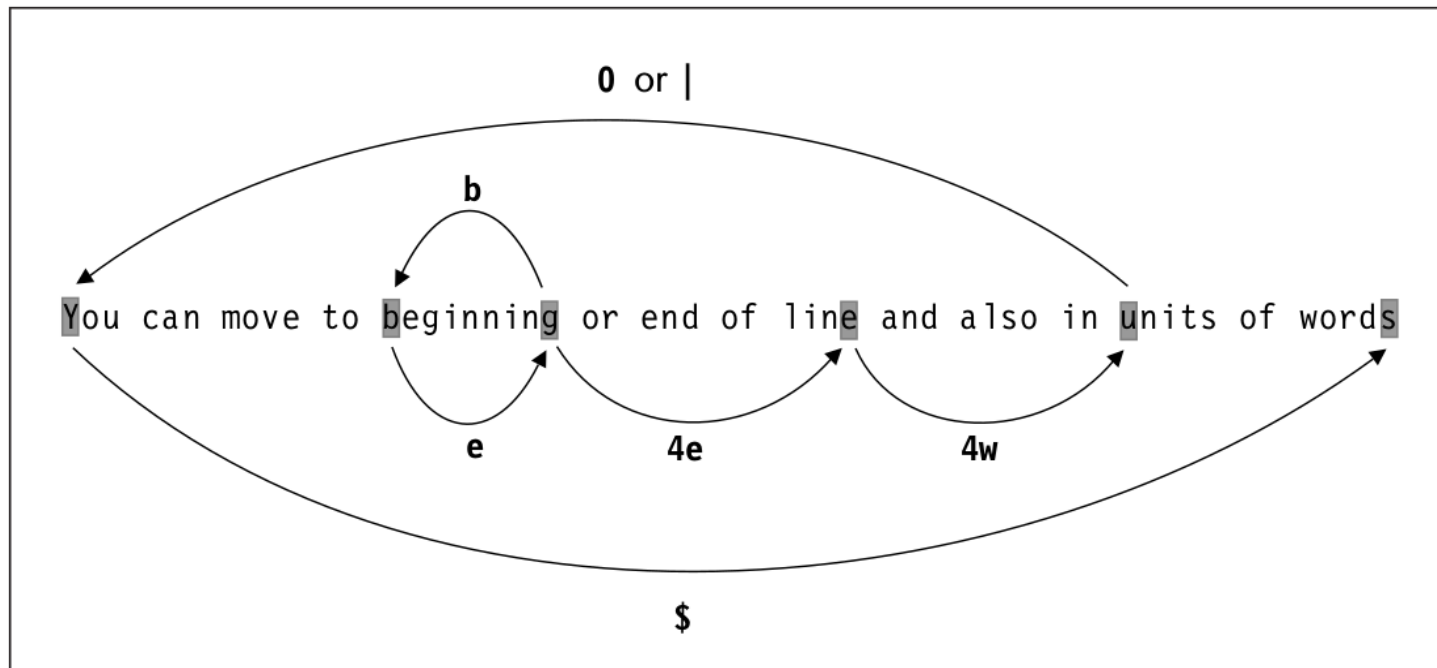
Scrolling	
[Ctrl-f]	Scrolls full page forward
5[Ctrl-f]	Scrolls five full pages forward
[Ctrl-b]	Scrolls full page back
[Ctrl-d]	Scrolls half page forward
[Ctrl-u]	Scrolls half page back
[Ctrl-l]	Redraws the screen (no repeat factor)

Word Navigation

b	Moves back to beginning of word
4b	Moves back four words to beginning of word
e	Moves forward to end of word
w	Moves forward to beginning of word
8w	Moves forward to beginning of 8th word



Finer Navigation with b , e , w , 0 and \$



Control Commands

- Following commands can be used with the Control Key to perform various functions

CTRL+d	Moves forward 1/2 screen	CTRL+y	Moves the screen down one line
CTRL+f	Moves forward one full screen	CTRL+u	Moves the screen up 1/2 page
CTRL+u	Moves backward 1/2 screen	CTRL+d	Moves the screen down 1/2 page
CTRL+b	Moves backward one full screen	CTRL+b	Moves the screen up one page
CTRL+e	Moves the screen up one line	CTRL+f	Moves the screen down one page

Editing Files

- To edit the file, you need to be in the insert mode.
- There are many ways to enter the insert mode from the command mode.
 - **i** Inserts text before the current cursor location
 - **I** Inserts text at the beginning of the current line
 - **a** Inserts text after the current cursor location
 - **A** Inserts text at the end of the current line
 - **o** Creates a new line for text entry below the cursor location
 - **O** Creates a new line for text entry above the cursor location

Original Text**vi Commands****Transformed Text**

```
printf(No entry);
```

```
i"[Esc]
```

```
printf("No entry);
```

```
printf("No entry);
```

```
i\n"[Esc]
```

```
printf("No entry\n");
```

```
if (x < 5) {
```

```
break;
```

```
i[Tab][Esc]
```

```
if (x < 5) {
```

```
break;
```

Original Text**vi Commands****Transformed Text**

```
echo "Filename:"
```

```
a\c[Esc]
```

```
echo "Filename:\b"
```

```
if [$x -gt 5 ]
```

```
a[Esc]
```

```
if [$x -gt 5 ]
```

```
case 
```

```
a $# in[Esc]
```

```
case $# in
```

```
echo ""
```

```
10a*[Esc]
```

```
echo "*****"
```

Using I and A and Opening a New Line with o and O

Original Text	vi Commands	Transformed Text
Set up an infinite loop	I /* [Esc]	/*Set up an infinite loop
/*Set up an infinite loop	A * /[Esc]	/* Set up an infinite loop */
exit(0)	A ; [Enter]][Esc]	exit(0); }

Original Text	vi Commands	Transformed Text
#count.sh: Counts patterns	o # [Esc]	#count.sh: Counts patterns #
#count.sh: Counts patterns	O #!/bin/sh [Esc]	#!/bin/sh #count.sh: Counts patterns #

Deleting Characters

- Here is a list of important commands, which can be used to delete characters and lines in an open file.
 - **x** Deletes the character under the cursor location
 - **X** Deletes the character before the cursor location
 - **dw** Deletes from the current cursor location to the next word
 - **d^** Deletes from the current cursor position to the beginning of the line
 - **d\$** Deletes from the current cursor position to the end of the line
 - **D** Deletes from the cursor position to the end of the current line
 - **dd** Deletes the line the cursor is on
- As mentioned above, **most commands in vi can be prefaced by the number of times you want the action to occur.** For example, **2x** deletes two characters under the cursor location and **2dd** deletes two lines the cursor is on.

Change Commands

- You also have the capability to change characters, words, or lines in vi without deleting them.
- Here are the relevant commands.
 - **cc** Removes the contents of the line, leaving you in insert mode.
 - **cw** Changes the word the cursor is on from the cursor to end of the word.
 - **r** Replaces the character under the cursor. vi returns to the command mode after the replacement is entered.
 - **R** Overwrites multiple characters beginning with the character currently under the cursor. You must use **Esc** to stop the overwriting.
 - **s** Replaces the current character with the character you type. Afterward, you are left in the insert mode.
 - **S** Deletes the line the cursor is on and replaces it with the new text. After the new text is entered, vi remains in the insert mode.

Copy and Paste Commands

- You can copy lines or words from one place and then you can paste them at another place using the following commands.
 - **yy** Copies the current line.
 - **yw** Copies the current word from the character the lowercase w cursor is on, until the end of the word.
 - **p** Puts the copied text after the cursor.
 - **P** Puts the yanked text before the cursor.

Advanced Commands

- There are some advanced commands that allow for more efficient use of vi.
 - **J** Joins the current line with the next one. A count of **j** commands join many lines.
 - **<<** Shifts the current line to the left by one shift width.
 - **>>** Shifts the current line to the right by one shift width.
 - **~** Switches the case of the character under the cursor.
 - **^G** Press Ctrl and G keys at the same time to show the current filename and the status.
 - **U** Restores the current line to the state it was in before the cursor entered the line.
 - **u** This helps undo the last change that was done in the file. Typing 'u' again will re-do the change.
 - **:f** Displays the current position in the file in % and the file name, the total number of file.
- **:f filename** Renames the current file to filename.
- **:w filename** Writes to file filename.
- **:e filename** Opens another file with filename.
- **:cd dirname** Changes the current working directory to dirname.
- **:e #** Toggles between two open files.
- **:n** In case you open multiple files using vi, use **:n** to go to the next file in the series.
- **:p** In case you open multiple files using vi, use **:p** to go to the previous file in the series.
- **:r file** Reads file and inserts it after the current line.
- **:nr file** Reads file and inserts it after the line **n**.

Word and Character Searching

- The vi editor has two kinds of searches: **string** and **character**. For a string search, the / and ? commands are used.
- When you start these commands, the command just typed will be shown on the last line of the screen, where you type the particular string to look for.
- These two commands differ only in the direction where the search takes place –
 - The / command searches forwards (downwards) in the file.
 - The ? command searches backwards (upwards) in the file.
- The **n** and **N** commands repeat the previous search command in the same or the opposite direction, respectively.

Word and Character Searching

- The character search searches within one line to find a character entered after the command. The **f** and **F** commands search for a character on the current line only. **f** searches forwards and **F** searches backwards and the cursor moves to the position of the found character.
- The **t** and **T** commands search for a character on the current line only, but for **t**, the cursor moves to the position before the character, and **T** searches the line backwards to the position after the character.

Running Commands

- The vi has the capability to run commands from within the editor.
- To run a command, you only need to go to the command mode and type `:! command`.
- For example, if you want to check whether a file exists before you try to save your file with that filename, you can type `:! ls` and you will see the output of **ls** on the screen.
- You can press any key (or the command's escape sequence) to return to your vi session.

Replacing Text

- The substitution command (**:s/**) enables you to quickly replace words or groups of words within your files.
- Following is the syntax to replace text
`:s/search/replace/g`
- The **g** stands for globally.
- The result of this command is that all occurrences on the cursor's line are changed.

Important Points to Note

- The following points will add to your success with vi –
 - You must be in command mode to use the commands. (Press Esc twice at any time to ensure that you are in command mode.)
 - You must be careful with the commands. These are case-sensitive.
 - You must be in insert mode to enter text.